

FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO

**RELATÓRIO DE MONITORAMENTO E ANÁLISE DE
DESEMPENHO COM IA**

Arthur Paltrinieri Silva

Lucas Kenichi Soares Abe

Pedro Dimitry Zyrianoff

Pedro Della Rosa Antônio

Projeto Interdisciplinar – Lideranças Empáticas

Inteligência Artificial e Aprendizado de Máquina

1. Introdução

O módulo de Inteligência Artificial do LiderAI foi desenvolvido para automatizar parte do processo de identificação e contagem de alimentos em campanhas de arrecadação. A proposta utiliza visão computacional com YOLO para reconhecer produtos em tempo real por meio de câmera, webcam ou celular conectado.

Nesta Entrega 02, o foco foi consolidar o treinamento do modelo, melhorar a execução local do detector e integrar a detecção ao sistema principal. O detector identifica o alimento, calcula o peso estimado, evita contagens duplicadas e envia o registro para o backend em nuvem.

2. Objetivo

O objetivo desta entrega foi treinar e utilizar um modelo YOLO capaz de reconhecer alimentos comuns em ações de arrecadação. Além da detecção visual, o módulo foi adaptado para registrar os resultados no sistema LiderAI por meio da API hospedada em nuvem.

3. Dataset utilizado

Foi utilizado um dataset próprio com 489 fotos. As imagens foram coletadas com variações de posição, distância, iluminação, ângulo e contexto, buscando aproximar o treinamento de situações reais de uso.

O conjunto de imagens foi organizado e anotado no formato YOLO. As classes representam alimentos e embalagens comuns de cesta básica, permitindo que o modelo identifique o alimento e, em alguns casos, diferencie pesos de embalagem.

4. Classes do modelo

As classes técnicas utilizadas no treinamento foram definidas para representar os produtos reconhecidos pelo detector. Posteriormente, no script local, essas classes são convertidas para nomes amigáveis exibidos ao usuário e registrados no sistema.

1kg_rice_package

5kg_rice_package

beans_package

fuba_package

oil_package

pasta_package

spaghetti_package

sugar_package

5. Anotação das imagens

As imagens foram classificadas com caixas delimitadoras, indicando a posição de cada alimento dentro da imagem. Esse processo é essencial em tarefas de detecção de objetos, pois o modelo aprende tanto a classe do objeto quanto sua localização.

Após a anotação, os arquivos foram organizados com uma pasta de imagens, uma pasta de rótulos no formato YOLO e um arquivo de classes.

6. Ambiente de treinamento

O treinamento foi realizado no Google Colab, utilizando GPU para acelerar o processamento. O uso do Colab permitiu treinar o modelo sem depender do poder computacional da máquina local.

Antes do treinamento, foi verificada a disponibilidade da GPU com o comando:

```
!nvidia-smi
```

7. Preparação dos dados

O dataset foi enviado ao Google Colab em formato compactado e descompactado no ambiente de execução. Em seguida, os dados foram separados em treinamento e validação, seguindo a estrutura esperada pela biblioteca Ultralytics.

A divisão utilizada foi de 90% para treinamento e 10% para validação. Considerando o total de 489 fotos, aproximadamente 440 imagens foram destinadas ao treinamento e 49 imagens à validação.

```
!unzip -q /content/data.zip -d /content/custom_data
```

```
!python train_val_split.py --datapath="/content/custom_data" --train_pct=0.9
```

8. Instalação das dependências

A biblioteca Ultralytics foi instalada no ambiente do Colab para permitir o treinamento e a validação do modelo YOLO.

```
!pip install ultralytics
```

9. Arquivo de configuração YAML

Antes do treinamento, foi criado um arquivo YAML contendo os caminhos das pastas de treino e validação, a quantidade de classes e os nomes das classes. Esse arquivo orienta o YOLO durante o treinamento.

```
path: /content/data
train: train/images
val: validation/images
nc: 8
names:
  - 1kg_rice_package
  - 5kg_rice_package
  - beans_package
  - fuba_package
  - oil_package
  - pasta_package
  - spaghetti_package
  - sugar_package
```

10. Modelo utilizado

O modelo utilizado foi o YOLO11n, identificado pelo arquivo yolo11n.pt. Essa versão foi escolhida por ser mais leve e adequada para execução

em tempo real, especialmente considerando que o detector precisa funcionar localmente com câmera.

Modelos menores tendem a ter maior velocidade de inferência, enquanto modelos maiores podem oferecer mais precisão, porém com maior custo computacional. Para a proposta do projeto, o YOLO11n apresentou um equilíbrio adequado entre desempenho e viabilidade de execução local.

11. Parâmetros de treinamento

O treinamento seguiu o modelo de referência baseado em Ultralytics YOLO no Google Colab, com ajuste para o modelo YOLO11n.

Modelo base: yolo11n.pt

Dataset: 489 fotos

Divisão: 90% treino e 10% validação

Resolução de treinamento: 640x640

Épocas: 60

```
!yolo detect train data=/content/data.yaml model=yolo11n.pt  
epochs=60 imgsz=640
```

12. Validação do modelo

Após o treinamento, o modelo foi testado com as imagens de validação. O objetivo dessa etapa foi verificar se o modelo conseguia identificar corretamente os alimentos e posicionar as caixas delimitadoras sobre os objetos.

```
!yolo detect predict model=runs/detect/train/weights/best.pt  
source=data/validation/images save=True
```

A validação permitiu observar visualmente os resultados e verificar se as classes estavam sendo reconhecidas de forma coerente.

13. Exportação do modelo treinado

Ao final do treinamento, o melhor peso gerado pelo YOLO foi salvo como best.pt. Esse arquivo foi copiado e renomeado para my_model.pt, facilitando sua utilização no detector local.

```
!mkdir /content/my_model
!cp /content/runs/detect/train/weights/best.pt
/content/my_model/my_model.pt
!cp -r /content/runs/detect/train /content/my_model
```

14. Execução local do detector

Como o Google Colab não possui acesso direto à câmera do computador do usuário, a inferência em tempo real foi executada localmente via VSCode. Para isso, o modelo treinado foi utilizado no script yolo_detect.py.

```
python yolo_detect.py --source usb0 --model my_model.pt --thresh 0.5
--resolution 1280x720
```

Para câmera externa ou DroidCam, pode-se alterar a origem para usb1:

```
python yolo_detect.py --source usb1 --model my_model.pt --thresh 0.5
--resolution 1280x720
```

15. Conversão de classes e cálculo de peso

O script local converte as classes técnicas do modelo em nomes amigáveis, como arroz, feijão, fubá, óleo, macarrão e açúcar. Além disso, cada classe possui um peso estimado associado, permitindo que o sistema some automaticamente o total arrecadado.

```
PESOS_CLASSES = {
    "1kg_rice_package": 1.0,
    "5kg_rice_package": 5.0,
    "beans_package": 1.0,
    "fuba_package": 0.5,
    "oil_package": 0.9,
    "pasta_package": 0.5,
    "spaghetti_package": 0.5,
```

```
"sugar_package":1.0  
}
```

16. Controle de duplicidade

Para evitar múltiplas contagens do mesmo alimento em frames consecutivos, foi implementada uma lógica simples de rastreamento. O sistema acompanha os objetos detectados e só contabiliza o alimento após ele permanecer visível por uma quantidade mínima de frames.

Essa estratégia reduz erros causados por oscilações da câmera, pequenas falhas de detecção ou permanência do mesmo item por vários frames.

17. Integração com o sistema LiderAI

Na Entrega 02, o detector foi integrado ao backend hospedado em nuvem. Ao iniciar, o script consulta a API para buscar os grupos cadastrados. Após selecionar o grupo, cada alimento detectado e contabilizado é enviado para o backend, que registra os dados no banco MySQL.

Detector de IA local

↓

Backend hospedado no Azure

↓

Banco MySQL hospedado na Aiven

Com isso, os registros gerados pela visão computacional podem ser visualizados nas telas de dashboard e histórico do sistema web.

18. Status final do módulo de IA

Dataset próprio com 489 fotos;

Imagens anotadas no formato YOLO;

Treinamento realizado no Google Colab com Ultralytics;

Modelo utilizado: YOLO11n;

Modelo exportado como my_model.pt;
Execução local com câmera, webcam ou DroidCam;
Conversão de classes técnicas para nomes amigáveis;
Cálculo automático de peso por alimento;
Controle de duplicidade por rastreamento simples;
Integração do detector ao backend em nuvem.

19. Conclusão

A Entrega 02 consolidou o módulo de visão computacional do LiderAI, transformando o treinamento do modelo em uma aplicação prática integrada ao sistema. O modelo YOLO11n foi treinado com um dataset próprio de 489 imagens e passou a ser utilizado em um detector local capaz de reconhecer alimentos em tempo real.

Além da detecção, o sistema calcula o peso estimado, reduz contagens duplicadas e envia os dados para o backend em nuvem. Dessa forma, o módulo de IA contribui diretamente para o objetivo do LiderAI: automatizar o acompanhamento de arrecadações e tornar os registros mais rápidos, organizados e confiáveis.